

# Advanced Programming Techniques

Modern programming languages, software design and AI-driven engineering

```
interface Course {  
    Design evolve(Code code);  
    Tests protect(Behavior b);  
    Spec guide(AIAgent agent);  
}
```

# Course information

Instructor and course repository.

## Pedro García López, PhD

Universitat Rovira i Virgili · Computer Engineering

### Office

238, ETSE

### Phone

977 558 510

### Email

[pedro.garcia@urv.cat](mailto:pedro.garcia@urv.cat)

•Pedro García López, PhD

–Office 238, ETSE

–Phone: 977 558 510

–Mail: [pedro.garcia@urv.cat](mailto:pedro.garcia@urv.cat)

### Repository

<https://github.com/pedrotgn/TAP>

# Course map

How the course is organized after the redesign.

## 1 • Foundations

Language concepts, paradigms, modern language evolution

## 2 • Design

Polymorphism, SOLID, refactoring, patterns

## 3 • Runtime power

Reflection, annotations, plugins, frameworks

## 4 • Quality

Testing, metrics, debugging, profiling, CI

## 5 • Modern systems

Services, cloud, serverless, distributed components

## 6 • AI engineering

Spec-driven engineering, AI-assisted coding, agents

**The course is code-first: concepts are introduced through small transformations of a running project.**

# Evaluation conditions

Three evaluation components with the same weight in the final score.

1/3

Individual AVA exam  
Without internet or AI · in the lab

1/3

Individual practical assignment  
Using spec-driven engineering

1/3

Theoretical exam  
Concepts, reasoning and design criteria

## Important rules

Minimum score of 4 in each part to pass the course.  
Only scores over 5 can be kept for future calls or retakes.  
The final average is computed only when all minimum thresholds are satisfied.

# Course philosophy

Advanced programming is learning how to transform code safely.

## Initial implementation

```
EmailService email = new EmailService();  
email.send(user, message);
```



## Designed implementation

```
class UserController {  
    private final NotificationService service;  
  
    UserController(NotificationService service) {  
        this.service = service;  
    }  
}
```

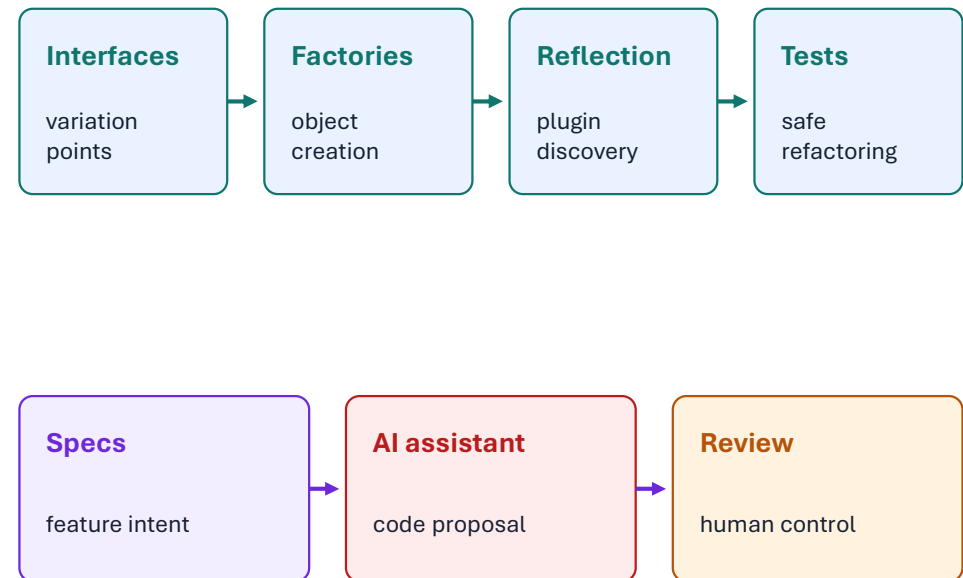
**The right abstraction is a tool for change, testing, reuse and AI-guided modification.**

# Running example

A modular report-processing application that evolves during the semester.

Week 1

```
public class App {  
    public static void main(String[] args) {  
        ReportGenerator generator = new  
        PdfReportGenerator();  
        generator.generate("students.csv");  
    }  
}
```



# Learning outcomes

What students should be able to do after the course.

- Read and reason about non-trivial code.
- Detect design problems and refactor safely.
- Use polymorphism, generics, reflection and annotations.
- Apply patterns and frameworks without over-engineering.
- Write tests that protect behavior.
- Use AI coding assistants critically.
- Write useful technical specifications and tasks.

Tests as a safety net

```
@Test
void jsonExporterProducesValidJson() {
    Report report = sampleReport();

    String json = exporter.export(report);

    assertThat(json).contains("\"students\"");
}
```

# Recommended bibliography and references

Books remain useful, but the course also uses modern documentation and repositories.

---

Design Patterns, Gamma et al.  
Refactoring, Martin Fowler.  
Effective Java, Joshua Bloch.  
Test Driven Development, Kent Beck.  
Modern Java documentation and GitHub examples.  
OpenSpec / Spec Kit documentation for spec-driven engineering.  
Selected articles on AI-assisted and agentic software engineering.

**The bibliography should evolve with the field: language features, tools and engineering practices are changing quickly.**



# Part I • Programming languages in 2026

From historical paradigms to modern engineering trade-offs.

# Why revisit programming languages?

Students already know basic definitions; now we use them as design criteria.

## Old framing

procedural vs object-oriented vs functional vs logic

## Modern framing

types, effects, memory, concurrency, deployment,  
tooling, AI support

## Course goal

choose the right abstraction and language feature  
for a software change

```
// Same problem, different design forces
users.stream().filter(User::active).toList(); // Java
users.iter().filter(|u| u.active).collect();  // Rust
users.filter(u => u.active)                    // TypeScript
```

# A compressed history of programming languages

The story is not a straight line: old ideas reappear in new ecosystems.



**Key idea: language evolution follows hardware, software architecture, developer productivity, and now AI tooling.**

# Paradigms are now mixed by default

Modern languages rarely belong to a single paradigm.

Java: OO + functional + immutable data

```
record User(String name, int age) {}

List<String> adults = users.stream()
    .filter(u -> u.age() >= 18)
    .map(User::name)
    .toList();
```

Rust: traits + functional iterators + ownership

```
trait Drawable { fn draw(&self); }

let names: Vec<_> = users.iter()
    .filter(|u| u.active)
    .map(|u| &u.name)
    .collect();
```

The relevant question is not “which paradigm?”  
It is “which design force does this construct help control?”  
State, variation, concurrency, reuse, verification, deployment.

# Static vs dynamic typing: the modern view

The boundary is more nuanced than in older slides.

## Static type checking

TypeScript / Java / Rust / Kotlin

```
function total(order: Order): Money {  
  return order.lines  
    .map(line => line.price)  
    .reduce(add);  
}
```



## Dynamic productivity

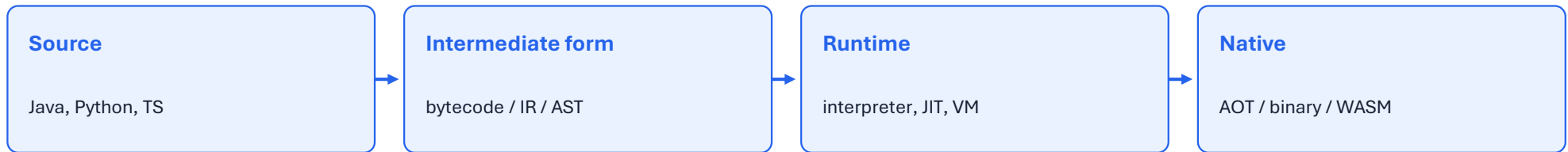
Python / JavaScript / Ruby

```
def total(order):  
    return sum(line.price for line in  
               order.lines)  
  
# often paired with tests, type hints,  
# linters
```

**Modern practice often combines both: gradual typing, type inference, linters, contracts and tests.**

# Compiled, interpreted, JIT, AOT: execution is a spectrum

The old compiled/interpreted dichotomy is too simple.



```
javac App.java      # source -> JVM bytecode
java App            # VM + JIT at runtime

python app.py       # interpreter + bytecode cache

rustc main.rs       # native binary

wasm-pack build     # WebAssembly target
```

# The type-system toolbox

Advanced type systems encode design decisions.

## Generics

List<User>  
Optional<T>  
Result<T,E>

## Traits / interfaces

Comparable  
Iterator  
Drawable

## Algebraic data

sealed classes  
enums  
pattern matching

## Null safety

Optional  
? / !  
non-null types

Modern Java example

```
sealed interface PaymentResult permits Accepted, Rejected {}  
record Accepted(String id) implements PaymentResult {}  
record Rejected(String reason) implements PaymentResult {}  
  
String message = switch (result) {  
    case Accepted a -> "OK " + a.id();  
    case Rejected r -> "Error: " + r.reason();  
};
```

# Memory and safety became language features

Recent languages respond to security, concurrency and performance needs.

```
// Rust ownership prevents use-after-free at compile time
fn print_name(name: String) {
    println!("{}", name);
}

let s = String::from("TAP");
print_name(s);
// println!("{}", s); // compile error: value moved
```

C/C++ give low-level control but require discipline.  
Garbage-collected languages improve productivity.  
Rust shows a third path: memory safety without a GC.  
Zig and Carbon explore modern systems-programming trade-offs.

**Advanced programming includes understanding the cost model and failure modes of the language runtime.**



# Concurrency models: threads are no longer the whole story

Modern languages expose different concurrency abstractions.

## Threads

shared memory  
locks  
race risks

## Async/await

non-blocking I/O  
promises/futures  
event loops

## Actors

message passing  
fault isolation  
Erlang/Elixir

## Data parallelism

streams  
Rayon  
GPU kernels

```
// Java virtual threads: familiar blocking style, lightweight execution
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    executor.submit(() -> service.fetchUser(id));
}
```

**Choosing a language also means choosing a concurrency model and failure model.**

# The modern language landscape

Languages are ecosystems: package managers, build tools, IDEs, CI and AI support matter.

## Backend / cloud

Java · Go · Kotlin · TypeScript · Python

## Systems

C · C++ · Rust · Zig · Carbon

## Data / AI

Python · Julia · R · Mojo

## Frontend / full stack

TypeScript · JavaScript · Dart

## Distributed / resilient

Erlang · Elixir · Go · Akka ecosystem

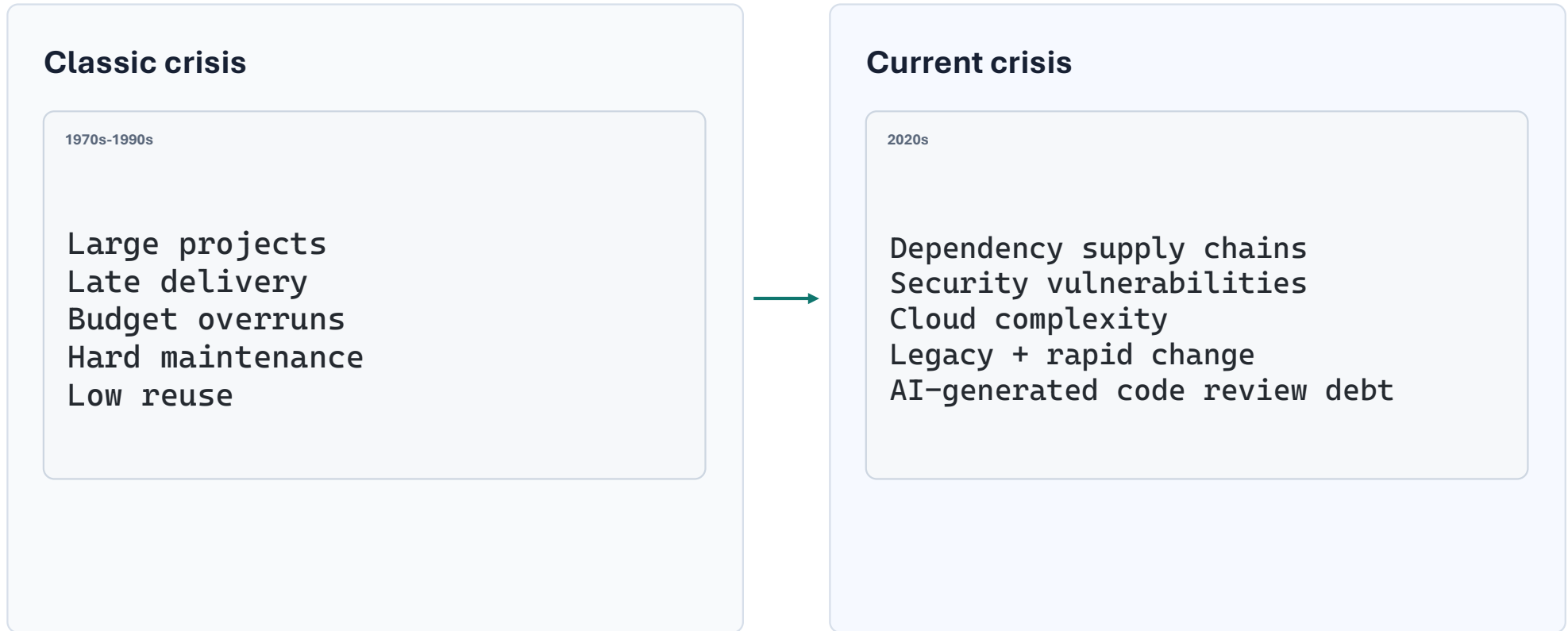
## Education / scripting

Python · JavaScript · Lua

**For this course we focus on transferable ideas, mainly with Java and the JVM ecosystem.**

# Software crisis, updated

The crisis did not disappear; it changed shape.



**The solution is still engineering discipline: modularity, tests, architecture, specifications and review.**

# From spaghetti code to dependency spaghetti

Today modularity problems often appear in dependencies and services.

## Old problem

```
// local spaghetti
public class OrderService {
  public void pay() {
    // SQL + email + payment + logging + PDF...
  }
}
```

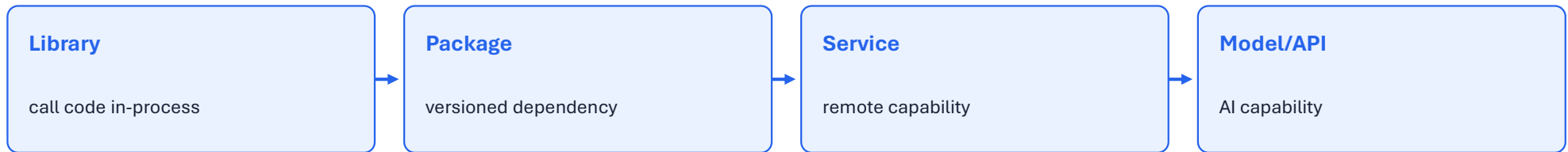
## New problem

```
# dependency spaghetti
app -> auth-service -> user-db
app -> payment-api -> risk-service
app -> queue -> worker -> storage
app -> npm package -> transitive dependency
```

Advanced design reduces coupling and makes dependencies explicit.  
Tests and contracts protect service boundaries.  
Specs make cross-component changes reviewable.

# Reuse evolved: libraries, packages, services, models

Software reuse is no longer only “software ICs”.



```
// Java: reuse through package + interface  
implementation("com.fasterxml.jackson.core:jackson-databind")  
  
ObjectMapper mapper = new ObjectMapper();  
String json = mapper.writeValueAsString(report);
```

**Reuse also creates risk: versioning, licensing, security, observability and vendor lock-in.**

# Software quality: external and internal views

The old quality attributes remain valid, but we add AI-era concerns.

## External quality

correctness · usability · performance · security · reliability

## Internal quality

modularity · readability · testability · low coupling · observability

## AI-era quality

spec clarity · generated-code reviewability · provenance · prompt/spec traceability

## Operational quality

deployability · monitoring · rollback · incident response · cost control

**Good code is not only code that works: it is code that can be understood, changed, verified and operated.**

# Maintenance and technical debt

Most cost is still paid after the first version works.

```
// technical debt is visible when change becomes
hard
if (customer.type().equals("PREMIUM")) {
    discount = 0.10;
} else if (customer.type().equals("STUDENT")) {
    discount = 0.15;
} else if (...) {
    ...
}
```

Maintenance = modification + debugging + adaptation.  
The cost of change depends on coupling and tests.  
AI can accelerate change, but also accelerate debt.  
Refactoring is a controlled investment in future change.

**Rule of thumb: code that is hard for a human to review is risky code to generate with AI.**

# Architecture levels, modernized

Architecture connects local design decisions with system-wide constraints.

## Micro architecture

classes, interfaces, patterns,  
modules

## Application architecture

layers, components, plugins,  
frameworks

## System architecture

services, queues, databases, APIs,  
observability

## Enterprise / platform

cloud, IAM, CI/CD, governance,  
compliance

```
// micro decision with architectural effects
public interface ReportExporter {
    boolean supports(Format format);
    void export(Report report, OutputStream out);
}
```

**A small interface can determine whether a system is extensible or rigid.**



# Micro-architecture: patterns and refactoring

Design patterns are vocabulary for recurring forces, not recipes to memorize.

```
// Strategy pattern: vary behavior without changing
client code
public interface PricingPolicy {
    Money priceFor(Order order);
}

public final class PremiumPricing implements
PricingPolicy {
    public Money priceFor(Order order) { ... }
}
```

Pattern names help communication.

Refactoring introduces patterns when the design needs them.

Overusing patterns creates accidental complexity.

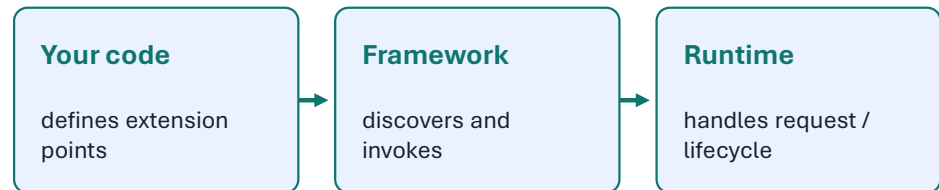
AI assistants often over-engineer: ask for the simplest design that satisfies the spec.

# Frameworks and inversion of control

Frameworks call your code; this is a central idea in modern development.

Spring-style example

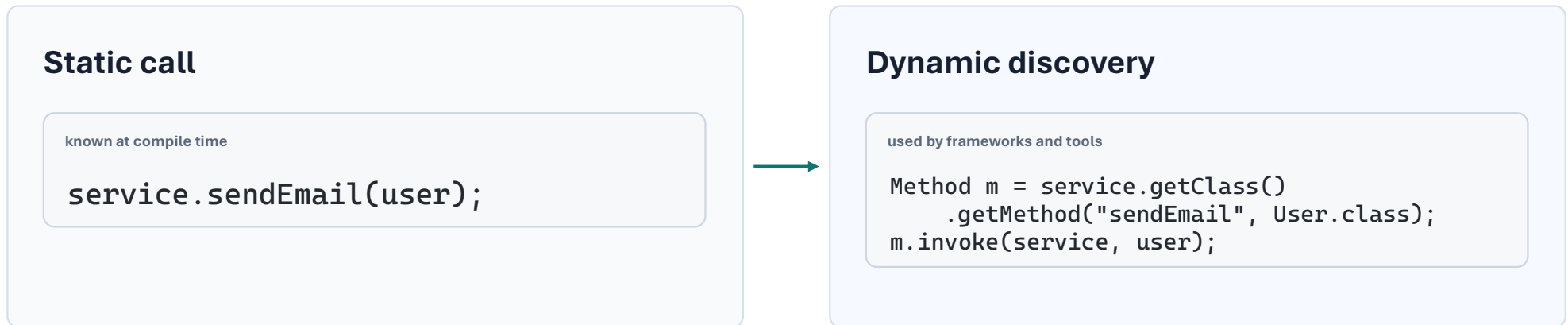
```
@RestController
class UserController {
    @GetMapping("/users/{id}")
    User find(@PathVariable Long id) {
        return service.find(id);
    }
}
```



Reflection, annotations, dependency injection and conventions make frameworks possible.

# Reflective programming in the age of frameworks

Reflection moved from exotic topic to everyday infrastructure.



Dependency injection containers discover constructors and annotations.

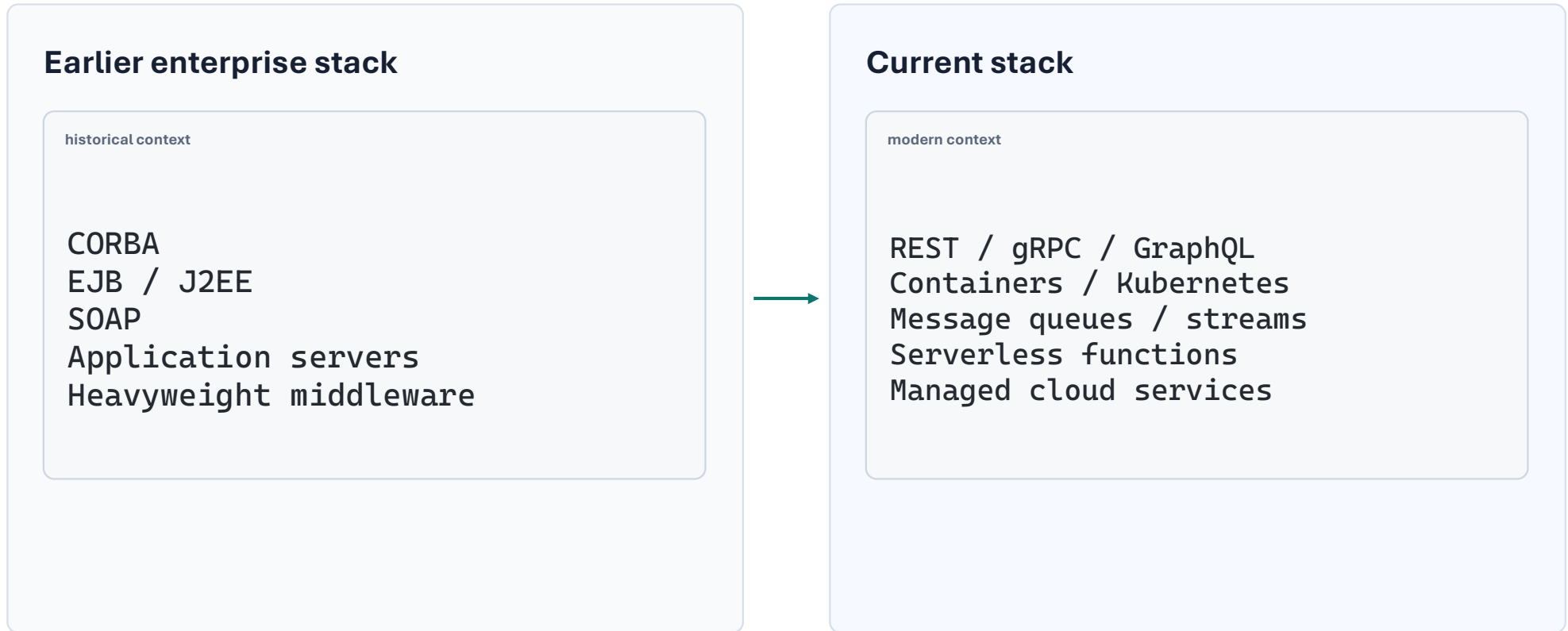
Serializers inspect fields and records.

Testing frameworks discover test methods.

Plugin systems load implementations dynamically.

# From CORBA/J2EE to cloud-native architectures

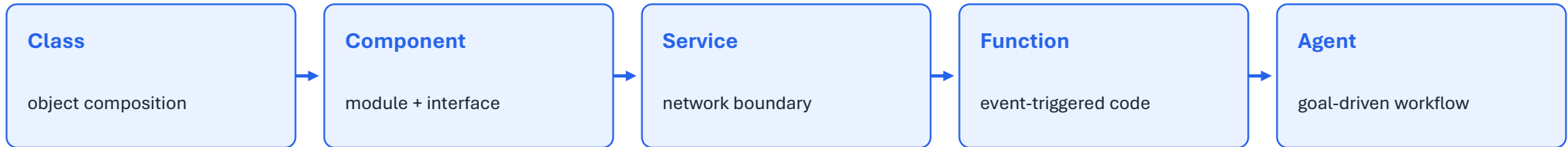
Keep the concepts, update the technologies.



**The persistent ideas are interfaces, contracts, interoperability, transactions, security and evolution.**

# From components to services to serverless

The unit of composition keeps moving upward.



```
// Event-driven function sketch
public void handle(S3Event event) {
    Report report = parse(event.objectKey());
    exporter.export(report, Format.JSON);
    queue.send(new ReportReady(report.id()));
}
```

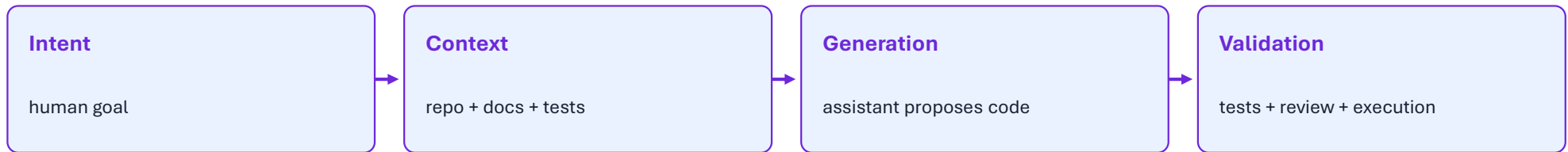
**Advanced programming must reason about boundaries: function call, process, container, service, workflow, agent.**

# **Part II · AI-assisted and spec-driven software engineering**

New tools make classical design discipline more important, not less.

# AI-assisted coding changes the workflow

AI is useful, but it changes where errors appear.



```
// Bad prompt
"Add JSON export"

// Better request
"Implement JsonReportGenerator using ReportGenerator.
Do not modify PdfReportGenerator.
Add CLI option --format json and tests for valid JSON and unknown formats."
```

**The quality of the prompt is limited by the quality of the design and specification.**

# Spec-driven engineering

Specifications become the coordination layer between humans, code and AI tools.

## Proposal

What changes and why?

## Design

How will the code structure change?

## Tasks

Which implementation steps are needed?

## Specs/tests

How do we validate behavior?

OpenSpec-style organization

```
changes/add-json-export/  
├── proposal.md  
├── design.md  
├── tasks.md  
└── specs/  
    └── report-export.md
```

Keep intent in the repository, not only in chat history.  
Review the change before implementation.  
Let AI execute tasks, not guess product intent.



# Example: proposal.md

A small spec reduces ambiguity before coding starts.

```
# Add JSON report export

## Motivation
Users need machine-readable reports for scripts and integrations.

## Proposed change
Add JsonReportGenerator implementing ReportGenerator.
Expose it with the CLI option: --format json.

## Non-goals
Do not modify PdfReportGenerator.
Do not change the Student model.
```

**A good proposal states intent, boundaries and non-goals.**

# Example: tasks.md

Tasks translate intent into reviewable implementation steps.

## # Tasks

- [ ] Create JsonReportGenerator
- [ ] Implement ReportGenerator
- [ ] Add CLI option --format json
- [ ] Add test for valid JSON output
- [ ] Add test for unknown format
- [ ] Ensure existing PDF tests still pass
- [ ] Update README examples

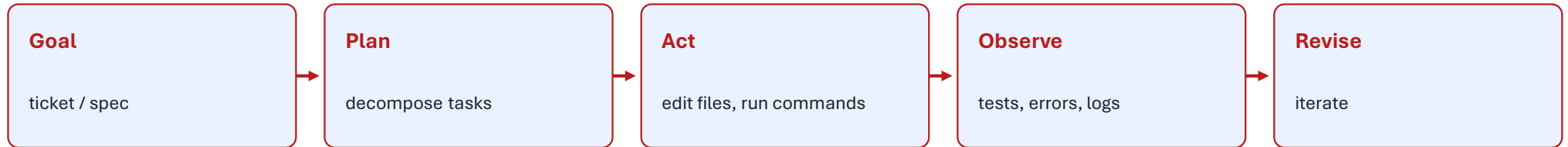
## Implementation target

```
public final class JsonReportGenerator implements
ReportGenerator {
    @Override
    public void generate(Report report,
        OutputStream out) {
        // serialize report as JSON
    }
}
```

**This is why specs connect directly with testing, refactoring and architecture.**

# Agentic programming

A coding agent is not just autocomplete; it plans, edits, runs tools and iterates.



```
agent.run("Implement add-json-export according to changes/add-json-export/tasks.md")

// Human still reviews:
// - architecture
// - security
// - tests
// - maintainability
// - unintended changes
```

**Agentic programming moves the human role toward specification, constraints, review and validation.**

# Risks of AI-generated code

AI accelerates both good engineering and bad engineering.

## Hallucinated APIs

calls functions that do not exist or misuse libraries

## Hidden coupling

changes unrelated files or creates fragile dependencies

## Security bugs

unsafe input handling, secrets, injection vulnerabilities

## Over-engineering

complex abstractions for simple problems

## Weak tests

tests that only confirm the generated implementation

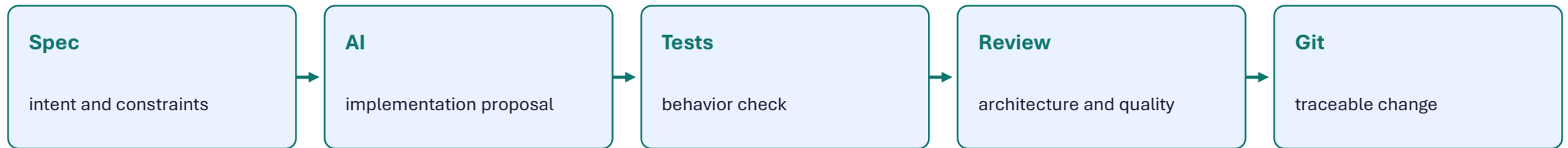
## Review debt

humans must understand code they did not write

**Course rule: generated code is not accepted unless you can explain, test and justify it.**

# Human-in-the-loop development

AI should be placed inside an engineering control loop.



Never rely only on generated explanations.  
Prefer small changes with focused tests.  
Use diffs and code review as the source of truth.  
Keep specifications, tests and decisions in the repository.

Control loop

```
git diff
mvn test
# review generated files before commit
git commit -m "Add JSON report export"
```

# Bridge to the rest of TAP

The rest of the course gives the tools needed to control change.

---

## Type systems

express constraints

## Polymorphism

support variation

## Refactoring

change design safely

## Patterns

share design vocabulary

## Reflection

enable frameworks and tools

## Testing

protect behavior

## Architecture

control dependencies

## Specs + AI

make change explicit

**Advanced programming is making software easier to change without losing control.**

# Summary

What students should remember from the updated introduction.

---

Programming languages are design tools shaped by hardware, architecture, safety and productivity.  
Paradigms are mixed in modern languages; focus on the problem each feature solves.  
Execution models, type systems, memory and concurrency matter for software architecture.  
The software crisis continues as dependency, cloud, security and AI review complexity.  
Spec-driven engineering helps humans and AI agree before coding.  
Agentic programming requires stronger tests, specs, review and architectural judgment.

**AI does not reduce the need for advanced programming techniques; it increases it.**

# Course motto

Code is not the only artifact.

---

Specifications  
Tests  
APIs  
Interfaces  
Architecture decisions  
Documentation  
Code reviews  
Operational dashboards

**Professional software is built from code plus the artifacts that make code understandable, changeable and safe.**

**Advanced programming is the discipline of making software easier to change without losing control.**



# Questions and discussion

What should a modern advanced programming course teach?

---

## For students

How do I become more than an AI prompt operator?

## For engineers

How do I design systems that can evolve safely?

## For teams

How do we combine specs, tests, review and AI agents?

**Next: type systems, polymorphism and the design of variation.**